Week 8: Machine Learning

Dr. Jamie Cockcroft

1 Introduction

> **Learning objectives**
>
> As a reminder, the learning objectives for this week are:
>
> 1. Explain how support vector machines construct decision boundaries for linear and non-linear data.
> 2. Understand how the hyperparameters C and gamma control model complexity.
> 3. Justify the use of train/test splits and cross-validation for model fitting and evaluation.
> 4. Interpret different evaluation metrics across classification contexts.
> 5. Fit, tune, and evaluate SVM classification models using the `caret` package in R.

In the lecture this week, we introduced Support Vector Machines (SVMs) as a supervised machine learning approach used for classification. We discussed how SVMs identify decision boundaries that separate classes, the role of margins, and how model behaviour is influenced by tuning parameters such as the cost function (C) and, for non-linear models, gamma. We also considered how model performance should be evaluated, particularly in the context of imbalanced data.

In today's practical, you will apply these concepts to two datasets. You will begin by working with a diabetes dataset, where you will fit and evaluate both linear and non-linear SVMs to identify whether or not someone has diabetes. You will then extend this analysis to a depression dataset. Across both datasets, you will split the data into training and test sets, apply cross-validation, and interpret model performance using measures such as accuracy, sensitivity, and specificity. Finally, if you're up for the challenge, you can see how much impact each tuning parameter has on your model's performance and how the number of outcomes influences predictions.

The aim of this practical is to help you develop a clear understanding of how SVMs behave, and how modelling decisions affect both performance and interpretation.

---

2 Setting up

2.1 Packages

Today's practical will use the following packages. You will need to install each one, before loading them at the top of your script.

```
library(tidyverse) # Data wrangling
library(caret)     # Machine learning functions
```

---

3 Part A: Core example

3.1 Introduction to the dataset

For the session we will be using the Pima Indians Diabetes dataset. This contains medical measurements of adult women of Pima Indian heritage. The data was originally compiled by the National Institute of Diabetes and Digestive and Kidney Diseases and are a commonly used example when learning support vector machines (SVM's).

To begin with, we should start by loading in the data:

```
# Load the diabetes dataset from GitHub
diabetes_data <- read_csv(
  "https://raw.githubusercontent.com/Premalatha-success/Datasets/main/pima-indians-diabetes-2.csv
) %>%

  # Lets rename some variables so they're a little clearer!
  rename(pregnancy = Preg,
         glucose = Plas,
         blood_pressure = Pres,
         tricep_thick = skin,
         insulin = test,
         BMI = mass,
         family_history = pedi,
         diagnosis = class) %>%

  # Some numeric variables are set to 0 but should be NA (missing!)
  mutate(
    across(
      c(glucose, blood_pressure, tricep_thick, insulin, BMI),
      ~ na_if(., 0)),

    # Need to also ensure diagnosis is a factor variable with two levels!
    diagnosis = factor(diagnosis,
```

```
                        # We want "diabetes" to be the predicted class
                        levels = c(1, 0),

                        # Labels cannot have spaces for caret
                        labels = c("Diabetes", "No_Diabetes")))
```

The aim is to predict whether an individual has diabetes based on clinical and demographic variables. It is a binary classification problem where they either have or do not have diabetes. Each row in the dataset represents one individual. The variables present in the data are:

- **`pregnancy`**: Number of times the patient has been pregnant.
- **`glucose`**: A measure of blood sugar levels after a glucose tolerance test. Higher values indicate poorer control of blood sugar.
- **`blood_pressure`**: Diastolic blood pressure measured in mm Hg (millimeters of mercury). Typically around 60 to 80.
- **`tricep_thick`**: A measure of body fat taken at the upper arm (measured in mm), providing an estimate of subcutaneous fat. Higher values indicate greater body fat.
- **`insulin`**: Amount of insulin in the blood two hours after consuming glucose. Very low suggests inadequate insulin production, while very high suggests insulin resistance.
- **`BMI`**: Body mass index. Higher values indicate larger body mass than expected given height.
- **`family_history`**: A measure of family history of diabetes. Higher values indicate a stronger genetic predisposition.
- **`age`**: Age in years.
- **`diagnosis`**: 0 = No diabetes, 1 = Diabetes present. This is the variable we are trying to predict.

Now that we have the data loaded in, the variables renamed, and the `NA`'s correctly specified, we should probably do a little exploration. Using your preferred R code, answer the following questions:

- How many patients in the dataset have at least one missing value (`NA`) in any variable?
  - Create a new dataset called `cleaned_diabetes` that includes only complete cases. How many patients remain after removing rows with missing values?
- Of the cleaned dataset, how many patients have diabetes? What proportion of the sample does this represent?
- Test whether patients with diabetes have significantly higher levels of `glucose` than those without diabetes.

```
# There are several ways you could do this, but here is one example...
# Count number of missing rows = 376 patients!
miss_count <- sum(!complete.cases(diabetes_data))

# Create cleaned dataset
cleaned_diabetes <- diabetes_data %>%
  drop_na()

# How many rows are left = 392 patients!
nrow(cleaned_diabetes)
```

```r
# How many patients have diabetes? 130 patients
# What proportion is this? 33%
cleaned_diabetes %>%
    count(diagnosis) %>%
    mutate(prop = n / sum(n))

# Predict glucose using diabetes classification
# Patients with diabetes have 33.8 units higher glucose than those without (p<.001).
mod = lm(glucose ~ diagnosis, data = cleaned_diabetes)
summary(mod)
```

3.2 Support Vector Machine

An SVM identifies a boundary that separates two groups. This boundary is chosen to maximise the margin, which is the distance between the boundary and the closest observations from each group. The observations closest to the boundary are called support vectors. These points determine where the boundary is placed. Maximising the margin improves how well the model generalises to new data.

3.2.1 Linear SVM

We begin with a linear SVM, where the boundary between groups is a straight line (or a flat hyperplane in higher dimensions).

Training and test sets

When building predictive models, it is essential to evaluate how well the model performs on data that were not used during training. If we fit and evaluate the model on the same dataset, the resulting performance estimates are likely to be overly optimistic. To avoid this, we divide the dataset into two parts. The training set is used to fit and tune the model, while the test set is reserved for evaluating final performance.

In this example, we assign 80% of the data to the training set and 20% to the test set.

```r
# To ensure reproducibility when run!
set.seed(123)

# Create an index of 80% of the data we should use for training
diabetes_train_index <- createDataPartition(cleaned_diabetes$diagnosis,
                                            p = 0.80,
                                            list = FALSE)

# Split the dataset according to the index
diabetes_train_data <- cleaned_diabetes[diabetes_train_index, ]
diabetes_test_data  <- cleaned_diabetes[-diabetes_train_index, ]
```

The `createDataPartition()` function performs a stratified split. This means that it samples separately within each outcome category so that the proportion of patients with and without diabetes is approximately preserved in both the training and test sets. This is particularly important because the dataset is imbalanced, with more cases of no diabetes than diabetes. You should verify that this stratification has worked as intended by comparing the proportions in the training and test sets with those in the original dataset using `diabetes_train_data %>% count(diagnosis)`.

Cross-Validation

In addition to using a test set, we also apply cross-validation within the training data to obtain a more reliable estimate of model performance. In 10-fold cross-validation, the training data are divided into 10 approximately equal parts. The model is trained on 9 parts and evaluated on the remaining part. This process is repeated 10 times so that each part is used once as a validation set. The performance is then averaged across all iterations.

```
diabetes_cv_control <- trainControl(
  method = "cv",            # k-fold cross-validation
  number = 10,              # k = 10 folds
  classProbs = TRUE         # Compute class probabilities
)
```

Cross-validation reduces the likelihood that our results depend on a particular split of the data and provides a more stable estimate of how the model is expected to perform on new data.

Fitting the model

We will now fit the model using the training set.

```
# Set for reproducible output
set.seed(123)

diabetes_svm <- train(
  diagnosis ~ .,                    # Model (use all variables)
  data = diabetes_train_data,       # Data
  method = "svmLinear",             # Linear SVM
  trControl = diabetes_cv_control,  # Cross-validation
  preProcess = c("center", "scale")  # Standardise data
)

diabetes_svm
```

```
Support Vector Machines with Linear Kernel

314 samples
  8 predictor
  2 classes: 'Diabetes', 'No_Diabetes'
```

```
Pre-processing: centered (8), scaled (8)
Resampling: Cross-Validated (10 fold)
Summary of sample sizes: 282, 282, 283, 283, 283, 283, ...
Resampling results:

  Accuracy   Kappa
  0.7864919  0.4724769

Tuning parameter 'C' was held constant at a value of 1
```

You should see from the output that 314 participants (samples) were used, with 8 predictors, and the model was attempting to classify whether or not someone had diabetes. Always check this matches what you intended! An unexpected number of predictors or samples usually points to an error somewhere. You specified `preProcess = c("center", "scale")`, so `caret` standardised the predictors before fitting. Our 10-fold cross-validation also appears to have run. The training data were split into 10 folds of approximately equal size.

The key performance metrics reported are accuracy and Cohen's kappa. Accuracy represents the proportion of correctly classified cases and in our case is 79%. However, when the classes are imbalanced, accuracy can give a misleading impression of performance. Cohen's kappa adjusts for the level of agreement that would be expected by chance. A kappa value around 0.47 would typically be interpreted as moderate agreement, indicating that the model performs better than chance but is far from perfect (if you're interested, you can read the Landis & Koch, 1977 paper on benchmarks for kappa). You want kappa to be $> .20$, and ideally $> .60$!

Take note that the `C` parameter (the cost function) was held constant at a value of 1. We will come back to this in Part C: Challenge yourself!.

Model predictions

We can now evaluate the performance of the SVM model on the independent test set (the 20% of responses we kept separate). This will provide a more realistic estimate of how the model performs on new, unseen data.

```r
# Generate predicted class labels for the test data
diabetes_test_pred <- predict(diabetes_svm, newdata = diabetes_test_data)

# Compare predictions with the observed outcome
diabetes_svm_cm <- confusionMatrix(
  data = diabetes_test_pred,
  reference = diabetes_test_data$diagnosis,
  positive = "Diabetes"
)

diabetes_svm_cm
```

```
Confusion Matrix and Statistics
```

```
            Reference
Prediction    Diabetes No_Diabetes
  Diabetes          15           6
  No_Diabetes       11          46

               Accuracy : 0.7821
                 95% CI : (0.6741, 0.8676)
    No Information Rate : 0.6667
    P-Value [Acc > NIR] : 0.01803

                  Kappa : 0.4848

 Mcnemar's Test P-Value : 0.33198

            Sensitivity : 0.5769
            Specificity : 0.8846
         Pos Pred Value : 0.7143
         Neg Pred Value : 0.8070
             Prevalence : 0.3333
         Detection Rate : 0.1923
   Detection Prevalence : 0.2692
      Balanced Accuracy : 0.7308

       'Positive' Class : Diabetes
```

The `predict()` function generates a predicted class for each patient in the test set based on our SVM model. The `confusionMatrix()` function then compares these predictions with the observed diabetes status of each patient.

The first part of the output tells us how well it predicted diabetes status across the left out patients. The model correctly identifies 15 patients with diabetes (true positives) and 46 patients without diabetes (true negatives). However, the model incorrectly classified 11 patients who actually had diabetes as not having it (false negative) and 6 patients who did not have diabetes as having it (false positives).

Overall, the accuracy of the model is 78%; this value is higher than the No Information Rate of 67%. The associated p-value is .018, which indicates that the model performs significantly better than this baseline. However, the 95% CI for accuracy ranges from 0.67 to 0.87. Because the lower bound is identical to the No Information Rate, this suggests some uncertainty in the estimate and indicates that the improvement over baseline performance is modest at best!

Next we can look at sensitivity, specificity, and balanced accuracy. The sensitivity is 0.58; this means that the model correctly identifies around 58% of patients who have diabetes. The specificity is 0.88; this indicates that about 88% of patients without diabetes are correctly identified. The model is therefore more accurate when identifying non-cases than cases. The balanced accuracy, which averages sensitivity and specificity, is 0.73; this provides a more appropriate summary of performance when the classes are imbalanced. Overall, the pattern of results shows that the model performs reasonably well in identifying patients without diabetes, but is less effective at identifying patients who do have the condition. The

relatively low sensitivity indicates that a substantial proportion of diabetes cases are being missed. In a clinical setting, this would be a concern, as false negatives may delay diagnosis and treatment.

3.2.2 Non-Linear SVM

The assumption of a linear boundary may be too restrictive for some datasets. Therefore, it may be a good idea to consider a non-linear SVM using a radial basis function kernel!

```
set.seed(123)

diabetes_svm_nl <- train(
  diagnosis ~ .,                    # Model (use all variables)
  data = diabetes_train_data,       # Data
  method = "svmRadial",             # Use Radial Basis Function
  trControl = diabetes_cv_control,  # Cross-validation
  preProcess = c("center", "scale")  # Standardise data
)

diabetes_svm_nl
```

```
Support Vector Machines with Radial Basis Function Kernel

314 samples
  8 predictor
  2 classes: 'Diabetes', 'No_Diabetes'

Pre-processing: centered (8), scaled (8)
Resampling: Cross-Validated (10 fold)
Summary of sample sizes: 282, 282, 283, 283, 283, 283, ...
Resampling results across tuning parameters:

  C     Accuracy   Kappa
  0.25  0.7705645  0.4714199
  0.50  0.7607863  0.4138166
  1.00  0.7574597  0.4041609

Tuning parameter 'sigma' was held constant at a value of 0.1292167
Accuracy was used to select the optimal model using the largest value.
The final values used for the model were sigma = 0.1292167 and C = 0.25.
```

Unlike the linear SVM, the radial model has two tuning parameters: `C`, which controls the penalty for misclassification, and `sigma` (gamma), which controls the shape of the radial basis function kernel. `caret` automatically tests a range of `C` values when doing a non-linear fit with cross-validation to see how it affects performance. Again, we will return to what these are doing in Part C: Challenge yourself!

Nevertheless, similar to the linear SVM output, we still get Accuracy and Kappa values. However, given that they are reported at each level of C, we should look for the one that produced the best fit. In this

case, it was when `C = 0.25`. Therefore, the radial SVM achieved an accuracy of around 77%, which is slightly below our linear model. This suggests that allowing a non-linear decision boundary is not leading to any further improvements in predictive performance for these data.

Before we make any decisions though, let's see how well the model performs on the `diabetes_test_data`. Use what you've learned above to generate a confusion matrix for the unseen data and evaluate performance based on sensitivity and specificity. Is the model doing any better?

```
# Generate predicted class labels for the test data
diabetes_test_pred_nl <- predict(diabetes_svm_nl, newdata = diabetes_test_data)

# Compare predictions with the observed outcome
diabetes_svm_nl_cm <- confusionMatrix(
  data = diabetes_test_pred_nl,
  reference = diabetes_test_data$diagnosis,
  positive = "Diabetes"
)

diabetes_svm_nl_cm
```

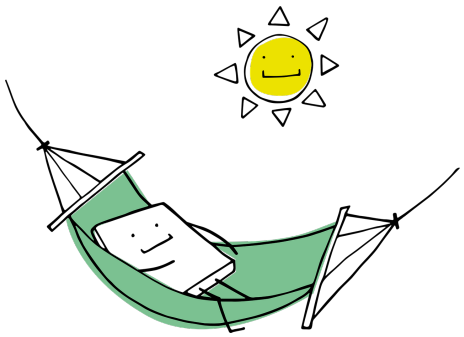
The first part of the output shows how well the non-linear model predicted diabetes status in the test set. The model correctly classified 17 patients with diabetes (true positives) and 42 patients without diabetes (true negatives). However, it incorrectly classified 9 patients with diabetes as not having it (false negatives) and 10 patients without diabetes as having it (false positives).

The overall accuracy of the model is 76%, which is slightly lower than the linear model using the same test set (78%). Although this is still higher than the No Information Rate of 67%, the associated p-value is .057, indicating that the model does not perform significantly better than simply predicting the majority class. Sensitivity is 0.65, meaning that only around 65% of patients with diabetes are correctly identified. This represents an increase compared to the linear model which was 58%. The specificity is 0.81, which is lower than the linear model (88%). The balanced accuracy is 73%, which is identical to the linear model.

Overall, the pattern of results suggests that the non-linear model performs better at classifying patients with diabetes than the linear model. However, it does make more errors when identifying people who don't have diabetes.

Take a break!

If you haven't already taken a break today, we suggest you do so before you start the next section.



---

4 Part B: Apply your knowledge

4.1 Introduction to the dataset

Next we will work with the `depression` dataset. The dataset contains data from an online study looking at what factors may best predict whether or not someone is currently experiencing depression, with participants being asked to complete a range of different measures. We should begin by loading in the data into R; this does assume that you have downloaded the data from the VLE and saved it in the appropriate `/data` folder in your project directory.

```
depression_data <- read_csv("../data/ambr8_depression-data.csv") %>%
  # We need to ensure depression is a factor and specify level order for caret
  mutate(depression = factor(depression,
                             levels = c("yes", "no")))
```

You should see the data is made up of many different columns.

- **`id`**: Unique participant identifier
- **`age`**: Age in years
- **`sex`**: Self-reported sex at birth
- **`ethnicity`**: Self-reported ethnicity
- **`education`**: Highest level of education completed
- **`employment_status`**: Current employment status
- **`region`**: UK region of residence
- **`financial_stress`**: Perceived financial strain, 0 to 10, higher scores indicate greater stress
- **`neighbourhood_deprivation`**: Area-level deprivation based on postcode, 0 to 10, higher scores indicate greater deprivation
- **`physical_activity_days`**: Number of days per week with physical activity
- **`alcohol_units_week`**: Weekly alcohol consumption in units
- **`sleep_hours`**: Average hours of sleep per night
- **`sleep_quality`**: Self-reported sleep quality (very poor to very good)
- **`screen_time_hours`**: Daily screen time in hours
- **`caffeine_cups_day`**: Daily caffeine intake in cups

- **chronic_pain**: Presence of a chronic pain condition, yes or no
- **family_history_depression**: Family history of depression, yes or no
- **past_anxiety_dx**: Previous clinical diagnosis of an anxiety disorder, yes or no
- **antidepressant_use**: History of antidepressant use, yes or no
- **therapy_history**: History of psychological therapy, yes or no
- **perceived_stress_scale**: General stress score, 0 to 40, higher scores indicate greater stress
- **gad7_total**: Anxiety score, 0 to 21, higher scores indicate greater anxiety
- **phq9_total**: Depression severity score, 0 to 27, higher scores indicate greater severity
- **wellbeing_score**: Self-reported wellbeing, 0 to 100, higher scores indicate better wellbeing
- **rumination_score**: Tendency to ruminate, higher scores indicate greater rumination
- **sleep_disturbance_score**: Sleep disturbance severity, higher scores indicate greater disturbance
- **GP_visits_last_year**: Number of GP visits in the past year
- **self_rated_health**: Self-rated overall health, 0 to 10, higher scores indicate better health
- **depression**: Current depression status

4.1.1 Explore the data

Before fitting any SVM, you should explore the dataset to understand the variables, check data quality, and identify potential issues. Use your preferred R code to answer the following questions:

- How many participants are included in the dataset?
- Is the outcome variable (`depression`) balanced or imbalanced?
- What is the mean and standard deviation of these variables when grouped by `depression`:
  - `age`
  - `financial_stress`
  - `neighbourhood_deprivation`
  - `physical_activity_days`
  - `screen_time_hours`
- What is the relationship between `perceived_stress_scale`, `rumination_score`, and `sleep_disturbance_scor` on the mental health measures `phq9_total` and `gad9_total`?

```
# There are several ways you could do this, but here is one example...
# How many participants?
nrow(depression_data) # 891 participants

# Outcome variable balance?
# 559 = no, 332 = yes
depression_data %>%
  count(depression)

# Descriptive statistics by depression group
desc_dep <- depression_data %>%
  # Group by depression variable
  group_by(depression) %>%

  # What variables are we interested in?
```

```r
  summarise(across(
    c(age, financial_stress, neighbourhood_deprivation,
      physical_activity_days, screen_time_hours),

    # What metrics do we want?
    list(
      mean = ~mean(.x, na.rm = TRUE),
      sd   = ~sd(.x, na.rm = TRUE))))

# Correlation between variables!
cor_dep <- depression_data %>%
  # Make sure we only look at the relevant variables
  select(perceived_stress_scale,
         rumination_score,
         sleep_disturbance_score,
         phq9_total,
         gad7_total) %>%

  # Compute the Pearson's correlation!
  cor(use = "complete.obs") %>%

  # Turn to dataframe so we can make it easier to read
  as.data.frame() %>%
  rownames_to_column("var1") %>%

  # Ensure we're in long format where each variable is a column
  pivot_longer(-var1, names_to = "var2", values_to = "correlation") %>%

  # Remove unnecessary correlations!
  filter(var1 %in% c("phq9_total", "gad7_total"),
         var2 %in% c("perceived_stress_scale", "rumination_score",
                     "sleep_disturbance_score"))
```

4.2 Fitting the SVM's

It is now your turn to fit a linear and non-linear (radial basis function) SVM to the data. Use the same workflow you used for the `cleaned_diabetes` dataset and adapt it to this `depression_data`. In doing so, address the questions listed below.

1. Split the `depression_data` into a training set (75%) and a test set (25%).
   - Did the split maintain the imbalance of the original data?
2. Fit two SVM models to the training data using the following kernels:
   - A linear kernel
   - A radial basis function kernel
     - Ensure you apply 10-fold cross-validation!

- Remember to remove `id` from the dataset as it shouldn't be all that informative to predicting the outcome.
  - Relatedly, reflect on whether all measures are needed for your model.

3. For each model, examine the cross-validates results:
   - Which model has the highest accuracy?
   - Which model has the highest kappa?
4. Use each model to then generate predictions on the test dataset and construct a confusion matrix for each.
   - Which model has the highest accuracy?
     - Is it significantly better than the no information rate?
   - Which model has the highest sensitivity?
   - Which model has the highest specificity?
   - Do the conclusions from the test match those of the cross-validation?
   - Which model appears to be better at generalising?

```
# Data splitting
# To ensure reproducibility when run!
set.seed(123)

# Create an index of 75% of the data we should use for training
dep_train_index <- createDataPartition(depression_data$depression,
                                       p = 0.75,
                                       list = FALSE)

# Split the dataset according to the index
dep_train_data <- depression_data[dep_train_index, ]
dep_test_data  <- depression_data[-dep_train_index, ]

# Cross-validation settings
dep_cv_control <- trainControl(
  method = "cv",            # k-fold cross-validation
  number = 10,              # k = 10 folds
  classProbs = TRUE         # Compute class probabilities
)

# Set for reproducible output
set.seed(123)

# Fit a linear model...
dep_svm_lin <- train(
  depression ~ .,                   # Model
  data = dep_train_data %>%         # Data (use all but id)
    select(-id),
  method = "svmLinear",             # Linear SVM
  trControl = dep_cv_control,       # Cross-validation
```

```r
  preProcess = c("center", "scale")  # Standardise data
)

# Set for reproducible output
set.seed(123)

# Fit a radial basis function model...
dep_svm_rbf <- train(
  depression ~ .,                # Model
  data = dep_train_data %>%          # Data (use all but id)
    select(-id),
  method = "svmRadial",              # Radial basis function SVM
  trControl = dep_cv_control,        # Cross-validation
  preProcess = c("center", "scale")  # Standardise data
)

# Look at the output of each model...
# Both models appear to be doing equally well:
# Linear: 76% accuracy (0.46 kappa)
# Non-linear: 77% accuracy (0.49 kappa)
dep_svm_lin
dep_svm_rbf

# Apply the models to the unseen data
# Generate predicted class labels for the test data
dep_pred_lin <- predict(dep_svm_lin, newdata = dep_test_data)
dep_pred_rbf <- predict(dep_svm_rbf, newdata = dep_test_data)

# Compare predictions with the observed outcome...
# Linear model...
dep_lin_cm <- confusionMatrix(
  data = dep_pred_lin,
  reference = dep_test_data$depression
)

# RBF model...
dep_rbf_cm <- confusionMatrix(
  data = dep_pred_rbf,
  reference = dep_test_data$depression
)

# Confusion matrix (linear model)
# Accuracy: 78% (p < .001, better than NIR of 63%)
# Sensitivity: 61%
# Specificity: 88%
# Balanced accuracy: 75%
# ... better at negative (no depression) than positive (depression) cases
```

```r
dep_lin_cm

# Confusion matrix (RBF model)
# Accuracy: 77% (p < .001, better than NIR of 63%)
# Sensitivity: 65%
# Specificity: 84%
# Balanced accuracy: 75%
# ... better at negative (no depression) than positive (depression) cases
dep_rbf_cm

# Non-Linear model is generally better at predicting depression, but only slightly.
```

---

5 Part C: Challenge yourself!

5.1 Cost (`C`) and Sigma

So far, we have fitted SVM models and evaluated their performance, but we have not yet considered how the model is being controlled. As discussed in the lecture, there are two key parameters that influence SVM behaviour: `C` and sigma (what most people call gamma). In linear models, you only have one of these parameters to worry about: `C` (cost function). `C` controls how much the model penalises errors in classification. Larger values of `C` place greater emphasis on correct classification of the training data. Smaller values of `C` allow more errors but favour a wider margin and simpler decision boundary. For non-linear SVMs, you have both `C` and sigma to contend with. Sigma controls how far a single observation influences the decision boundary. Larger values of sigma will mean the influence of each observation is very local, making the decision boundary far more complex. In contrast, when sigma is small, the influence of each observation is broader, leading to smoother and more general decision boundaries.

In practice, researchers often examine how different values of `C` and `sigma` (what most people call gamma) affect model performance. In this section, you will do the same with the `depression_data`. To achieve this, you will need to consult the documentation for the `train()` function, paying particular attention to the `tuneGrid` argument, and work out how to specify your own tuning values.

> Click for a hint!
>
> There is a very helpful tutorial by the Ubiqum Code Academy that explains how `tuneGrid` works. Although it uses k-nearest neighbours as an example, the same logic applies here.

```r
# Create a grid of tuning parameter values to test
# expand.grid() is needed when you have multiple parameters so all combinations are created
svm_lin_grid <- expand.grid(
  C = c(0.01, 0.1, 1, 10)
)

# Now fit the linear SVM with the tuning parameters
```

```r
dep_svm_lin_tuned <- train(
  depression ~ .,                    # Model
  data = dep_train_data %>%          # Data (not id)
    select(!id),
  method = "svmLinear",              # Linear SVM
  trControl = dep_cv_control,        # Cross-validation
  preProcess = c("center", "scale"), # Standardise data
  tuneGrid = svm_lin_grid            # Use the defined tuning parameters (C)
)

# How does C affect training performance?
# We get 77% accuracy (0.49 kappa)
dep_svm_lin_tuned

# How does the new model do with unseen data?
dep_pred_lin_tuned <- predict(dep_svm_lin_tuned, newdata = dep_test_data)
dep_lin_cm_tuned <- confusionMatrix(
  data = dep_pred_lin_tuned,
  reference = dep_test_data$depression
)

# How does the model perform...?
# Accuracy: 78% (p < .001, better than NIR of 63%)
# Sensitivity: 66%
# Specificity: 86%
# Balanced accuracy: 76%
# ... better at negative (no depression) than positive (depression) cases
dep_lin_cm_tuned

# Let's repeat the above, but for the RBF
# ... C is less sensitive, so we test a wider range of values
# sigma is sensitive, so we use a smaller, more conservative range
svm_rbf_grid <- expand.grid(
  C = c(0.01, 0.1, 1, 10),
  sigma = c(0.001, 0.01, 0.1, 1)
)

# Now fit the linear SVM with the tuning parameters
dep_svm_rbf_tuned <- train(
  depression ~ .,                    # Model (use all except id)
  data = dep_train_data %>%          # Data (not id)
    select(-id),
  method = "svmRadial",              # RBF SVM
  trControl = dep_cv_control,        # Cross-validation
  preProcess = c("center", "scale"), # Standardise data
  tuneGrid = svm_rbf_grid            # Use the defined tuning parameters (C and sigma)
)
```

```r
# How does C affect training performance?
# We get 77% accuracy (0.48 kappa)
dep_svm_rbf_tuned

# How does the new model do with unseen data?
dep_pred_rbf_tuned <- predict(dep_svm_rbf_tuned, newdata = dep_test_data)
dep_rbf_cm_tuned <- confusionMatrix(
  data = dep_pred_rbf_tuned,
  reference = dep_test_data$depression
)

# How does the model perform...?
# Accuracy: 77% (p < .001, better than NIR of 63%)
# Sensitivity: 64%
# Specificity: 85%
# Balanced accuracy: 74%
# ... better at negative (no depression) than positive (depression) cases
dep_rbf_cm_tuned
```

5.2 Moving beyond two classes

As noted in the lecture, SVMs are fundamentally binary classifiers. They find a boundary between two groups. When there are more than two classes, the problem must be broken down into a series of binary problems. There are two common strategies for this: one-vs-one and one-vs-all. When using `caret` with `svmLinear`, the underlying `kernlab` engine uses **one-vs-one** by default. You do not need to specify this; it is handled automatically when the outcome variable has more than two levels.

We can look at this by using the `depression_data` again. However, we will need to create three labels rather than two. Specifically, we will use `phq9_total` to create a diagnostic category variable, then build an SVM to predict that category from the remaining variables in the dataset.

5.2.1 Create the diagnostic categories

We will collapse the PHQ-9 total scores into three groups based on established clinical thresholds: scores of 0 to 9 as "None", 10 to 19 as "Moderate", and 20 to 27 as "Severe".

```r
# Create a new variable "dep_diagnosis" based on PHQ-9 clinical thresholds.
depression_data <- depression_data %>%
  mutate(
    dep_diagnosis = factor(case_when(
      phq9_total <= 9  ~ "None",
      phq9_total <= 19 & phq9_total > 9 ~ "Moderate",
      phq9_total >= 20 ~ "Severe"
    ),
    levels = c("None", "Moderate", "Severe"))
  )
```

You should check whether the groupings are balanced. You should see that most people are classified as moderate.

5.2.2 Fit the model

Do what you have done before and fit a linear support vector machine to the data, but ensure you remove the `phq9_total` variable before fitting. Since `dep_diagnosis` was derived directly from `phq9_total`, including it as a predictor would give the model a variable that perfectly determines the outcome.

```
# To ensure reproducibility when run!
set.seed(123)

# Create an index of 75% of the data we should use for training
depmulti_train_index <- createDataPartition(depression_data$dep_diagnosis,
                                            p = 0.75,
                                            list = FALSE)

# Split the dataset according to the index
depmulti_train_data <- depression_data[dep_train_index, ]
depmulti_test_data  <- depression_data[-dep_train_index, ]

# Cross-validation settings
depmulti_cv_control <- trainControl(
  method = "cv",            # k-fold cross-validation
  number = 10,              # k = 10 folds
  classProbs = TRUE         # Compute class probabilities
)

# Set for reproducible output
set.seed(123)

# Fit a linear model...
# 70% accuracy (0.40 kappa)
depmulti_svm_lin <- train(
  dep_diagnosis ~ .,                  # Model
  data = depmulti_train_data %>%      # Data (exclude id and phq9)
    select(-id, -phq9_total),
  method = "svmLinear",               # Linear SVM
  trControl = depmulti_cv_control,    # Cross-validation
  preProcess = c("center", "scale")  # Standardise data
)

# Generate model predictions
testmulti_preds <- predict(depmulti_svm_lin, newdata = depmulti_test_data)

# Now the confusion matrix
depmulti_lin_cm <- confusionMatrix(testmulti_preds,
```

```
                                    depmulti_test_data$dep_diagnosis)

depmulti_lin_cm
```

The output for the confusion matrix will look a little different from what you have seen so far. With three classes, it is now a 3x3 table rather than 2x2. Take a moment to look at the structure.

The first thing to notice is the top row of the confusion matrix. The model never predicts "None". Every single case is assigned to either "Moderate" or "Severe". All 15 people who genuinely had no depression were misclassified as Moderate. This is a clear illustration of how class imbalance can cause problems. The "None" group makes up ~7% of the test data. Therefore, predicting "None" is rarely correct, so the model simply ignores it. It can still achieve 64% accuracy while being completely blind to an entire category. This is precisely why we need to look beyond accuracy and examine sensitivity and specificity for each class.

With three classes, `caret` reports sensitivity and specificity separately for each one. This works on a one-vs-all basis: for each class, sensitivity asks "of the people who belong to this group, how many did the model correctly identify?", and specificity asks "of the people who do not belong to this group, how many did the model correctly exclude?"

Let's work through each class.

None

Sensitivity is 0%. Of the 15 people who had no depression, the model identified zero of them. Specificity is 100%, but this does not indicate good performance. It simply means the model never *falsely* predicts None. Since it never predicts None at all, there are no false positives for this class.

Moderate

This is the class the model relies on most heavily. Sensitivity is 84%, meaning it correctly identified 98 of the 117 cases of moderate depression. It missed 19, placing them into the Severe category instead. However, the specificity of 42% means that, of the 105 people who were *not* Moderate (the None and Severe groups combined), the model labelled 61 of them as Moderate incorrectly. The model is over-predicting this category.

Severe

The model correctly identifies about half of the Severe cases: 44 out of 90. The other 46 are misclassified as Moderate. Specificity is much higher at 86%, meaning that when someone is not genuinely Severe, the model usually avoids labelling them as such. However, it misses half of the people who actually need the most urgent clinical attention!

Overall

The model defaults to Moderate for most cases. It is cautious about predicting Severe and completely ignores the None category. This follows the class sizes: Moderate is the largest group (53%), Severe is the next largest (41%), and None is very small (7%). The model has learned that predicting Moderate is the safest option because it is right more often than not. The cost is that it fails to distinguish meaningfully at the extremes! It could be worth seeing how much the `C` parameter affects things…

---

6 Summary

6.1 Recap

Today we've seen how support vector machines can be used to classify outcomes by identifying decision boundaries between groups, and how this can be extended from simple linear boundaries to more flexible non-linear ones when the data require it. We also saw how this approach can be implemented in practice using the `caret` package in R, giving us a full workflow for fitting, tuning, and evaluating SVM models across different classification problems.

During the practical, we also encountered some of the key decisions and challenges involved in using this technique. In particular, we saw why train and test splits, alongside cross-validation, are important for producing more trustworthy estimates of model performance, and why accuracy alone is often not enough when classes are imbalanced. You've also considered how tuning parameters such as `C` and sigma (gamma) affect model complexity and performance, and how different evaluation metrics can lead us to different conclusions about which model is most useful in context.

6.2 Further learning

The core reading for this week comes from Daniel Baker's textbook, specifically Chapter 14: Multivariate pattern analysis. If you want to gain more interactive experience in fitting support vector machines in R, there is also a relevant chapter on DataCamp. I would also recommend looking at the interactive demo by Jonas Greitemann that allows you to see how data points impact the decision boundary.

! Feedback!

These practical sessions are new this year. Please take a moment to rate the difficulty and length of these practical activities via this survey, and let us know of any issues you encountered or aspects you didn't understand (positive feedback is also welcome!). You will need to be logged in with your university account to respond, but your submission will be anonymous.
For more general questions about the practical, please post them on the VLE Discussion Board.